

A close-up photograph of a person's hand holding a thin, clear plastic strip. The hand is positioned on the right side of the frame, with fingers curled around the strip. The background is a solid, vibrant blue. The lighting is bright, casting soft shadows on the skin.

IMAGE CLASSIFICATION · PROJECT VIVA

IndiSign: recognizing hand-sign letters with a CNN

From raw sign photos to a tested prediction pipeline.

01 · THE TASK

The project turns a hand photo into a letter prediction

Goal: predict which letter a hand-sign photo represents.

Two project goals: build image preprocessing and a model that predicts the sign.

01

Input

Hand photo

02

Model

CNN learns
patterns

03

Output

Letter
prediction



The dataset contains 4,972 images across 24 sign classes

4,972

images

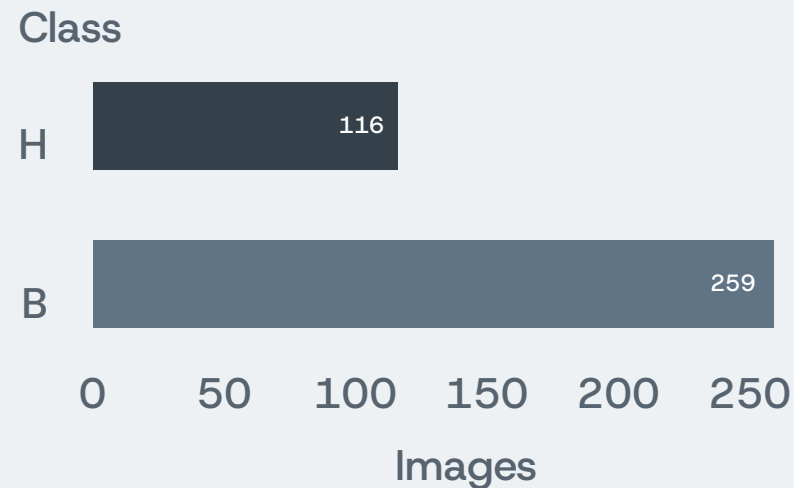
24

sign classes

Folders such as A, B, C become numeric labels for model training.

Integrity: every image is opened and verified; no corrupt files are reported.

Balance: class counts range from H with 116 images to B with 259, a 2.23:1 imbalance ratio.



Preprocessing makes every image comparable

One repeatable transformation turns varied photos into model-ready tensors.

01

Convert

Convert each image to RGB.

02

Resize

Resize it to 128 × 128 pixels.

03

Normalize

Normalize pixel values from 0–255 to 0–1.

04

Store

Store images as X with shape (images, 128, 128, 3), labels as y, and original paths for traceability.

Stratified splitting protects the evaluation

70%

Train

Learn visual patterns.

15%

Validation

Guide tuning and model selection.

15%

Test

Final held-out estimate.

Stratification keeps each sign represented in the same proportions.

Class weights give rarer signs more influence during training.

Augmentation adds variation without changing the sign

Training images receive random rotation, shifts, and zoom.

01

Rotate

Adds angle variation.

02

Shift

Changes position.

03

Zoom

Changes framing.

No flip: Flipping is intentionally excluded because a mirrored hand may not represent the same valid sign.

Result: the model is trained for changes in angle, position, and framing rather than memorizing one pose.

The baseline CNN learns from edges to finger patterns

01

Input

Augmented 128 × 128 RGB image.

02

Feature extraction

Three Conv2D + MaxPooling rounds move from edges to shapes to finger patterns.

03

Decision

Flatten → Dense(128) → Dropout(0.5) → Softmax with 24 outputs.

04

Training

Adam optimizer with sparse categorical cross-entropy.

Edges

Shapes

Fingers

Training controls overfitting and keeps the best checkpoint

01

Monitor

Early stopping halts training when validation loss stops improving.

02

Adapt

ReduceLROnPlateau lowers the learning rate when progress stalls.

03

Preserve

ModelCheckpoint preserves the best version.

30

baseline training cap · epochs

Best checkpoint means the final model is the strongest saved version, not simply the last version trained.

Random search tests the choices that matter most

SEARCH SPACE

Five choices

- Convolution filters
- Dense units
- Dropout
- Learning rate
- Batch size

TRADE-OFF

8

random trials

Instead of evaluating all 96 combinations.

SELECTION RULE

Validation accuracy

The winning settings are selected using validation accuracy, not the test set.

Principle: search the decisions that affect learning, then keep the test set untouched.

Evaluation looks beyond a single accuracy number

Model comparison

Compare baseline and final models on validation and test sets.

Per-sign metrics

Use precision, recall, and F1 to see whether performance is consistent across signs.

Confusion matrix

Read the confusion matrix, weakest F1 classes, most confused pairs, and misclassified images to understand failure modes.

Misclassified images

Understand failure modes.